SQL 실습예제 3장 0409 실습예제3_문제풀이

문제 20.[극장데이터베이스]

■ 테이블 구조와 데이터

극장

극장번호	극장이름	위치
1	롯데	잠실
2	메가	강남
3	대한	잠실

상영관

극장번호	상영관번호	영화제목	가격	좌석수
1	1	어려운 영화	15000	48
1	1	멋진 영화	7500	120
3	2	재밌는 영화	8000	110

예약

극장번호	상영관번호	고객번호	좌석번호	날짜
3	2	3	15	2025/09/01
3	1	4	16	2025/09/01
1	1	9	48	2025/09/01

고객

고객번호	이름	주소
3	홍길동	강남
4	김철수	잠실
9	박영희	강남

제약조건
영화가격은 20,000원을 넘지 않아야 한다.
상영관번호는 1~10사이다
같은 사람이 같은 좌석번호를 두 번 예약하지 않아야 한다.

기존 테이블이 있다면 먼저 삭제하자
DROP TABLE 예약;dnl
DROP TABLE 상영관;
DROP TABLE 고객;
DROP TABLE 극장;

(1)단순질의
①모든 극장의 이름과 위치를 보이시오

```
SELECT 극장이름, 위치 FROM 극장;
```

②'잠실'에 있는 극장을 보이시오

```
SELECT *  FROM 극장  WHERE 위치 = '잠실';
```

③'잠실'에 사는 고객의 이름을 오름차순으로 보이시오

```
SELECT 이름 FROM 고객 WHERE 주소 = '잠실' ORDER BY 이름 ASC;
```

④가격이 8,000원 이하인 영화의 극장번호,영화제목을 보이시오

```
SELECT 극장번호, 영화제목 FROM 상영관 WHERE 가격 <= 8000;
```

⑤극장 위치와 고객의 주소가 같은 고객을 보이시오

```
SELECT 고객.고객번호, 고객.이름, 고객.주소 FROM 고객
        JOIN 극장 ON 고객.주소 = 극장.위치;
```

(2)집계질의

①극장의 수는 몇 개인가? 3

```
SELECT COUNT(*) AS 극장수 FROM 극장;
```

②상영되는 영화의 평균 가격은 얼마인가? 10166.67

```
SELECT AVG(가격) AS 평균가격 FROM 상영관;
```

③2025년 9월 1일에 영화를 관람한 고객의 수는 얼마인가?

```
SELECT COUNT(DISTINCT 고객번호) AS 고객수  FROM 예약
      WHERE 날짜 = TO_DATE('2025/09/01', 'YYYY/MM/DD');
```

④전체 상영관의 총 좌석 수를 구하시오.

```
SELECT SUM(좌석수) AS 총좌석수
FROM    상영관;
```

⑤전체 영화 가격의 평균을 구하시오

```
SELECT SUM(좌석수) AS 총좌석수 FROM    상영관;
```

⑥상영관 테이블에서 가장 비싼 영화 가격과 가장 저렴한 영화 가격을 구하시오.

```
SELECT MAX(가격) AS 최고가격, MIN(가격) AS 최저가격  FROM    상영관;
```

⑦ 전체 예약 건수를 구하시오

```
SSELECT COUNT(*) AS 총예약건수 FROM    예약;
```

⑧극장 테이블에 등록된 극장의 총 개수를 구하시오

```
SELECT COUNT(*) AS 극장수 FROM   극장;
```

⑨ 상영관 테이블에서 가격이 10,000원 이상인 영화의 수를 구하시오

```
SELECT COUNT(*) AS 고가영화수 FROM   상영관 WHERE  가격 >= 10000;
```

(3)부속질의

①'대한'극장에서 상영된 영화 제목을 보이시오

```
SELECT 상영관.영화제목 FROM 상영관
    JOIN 극장 ON 상영관.극장번호 = 극장.극장번호
      WHERE 극장.극장이름 = '대한';
```

②'대한'극장에서 영화를 본 고객의 이름을 보이시오

```
SELECT DISTINCT 고객.이름 FROM 고객
    JOIN 예약   ON 고객.고객번호 = 예약.고객번호
    JOIN 상영관 ON 예약.극장번호 = 상영관.극장번호
              AND 예약.상영관번호 = 상영관.상영관번호
    JOIN 극장   ON 상영관.극장번호 = 극장.극장번호
             WHERE 극장.극장이름 = '대한';
```

③'대한 극장의 전체 수입을 보이시오

```
  SELECT 극장.극장이름, SUM(상영관.가격) AS 전체수입 FROM 예약
      JOIN 상영관 ON 예약.극장번호 = 상영관.극장번호
             AND 예약.상영관번호 = 상영관.상영관번호
      JOIN 극장   ON 상영관.극장번호 = 극장.극장번호
      WHERE 극장.극장이름 = '대한'
      GROUP BY 극장.극장이름;
```

④평균 가격보다 비싼 영화를 상영하는 상영관 번호와 영화제목을 구하시오

```
SELECT 상영관번호, 영화제목, 가격 FROM   상영관
WHERE  가격 > (SELECT AVG(가격) FROM 상영관);
```

⑤예약이 한 건 이상 있는 극장의 극장이름을 구하시오

```
SELECT 극장이름 FROM   극장
WHERE  극장번호 IN (    SELECT 극장번호 FROM 예약);
```

⑥가장 좌석수가 많은 상영관에서 상영 중인 영화를 구하시오

```
SELECT 영화제목, 좌석수 FROM   상영관
WHERE  좌석수 = (
    SELECT MAX(좌석수) FROM 상영관 );
```

⑦'강남'에 거주하는 고객이 예약한 좌석번호를 모두 구하시오

```
SELECT 좌석번호, 날짜 FROM   예약
WHERE  고객번호 IN (
    SELECT 고객번호
    FROM   고객
    WHERE  주소 = '강남'
);
```

⑧예약한 적이 없는 고객의 이름을 구하시오.

```
SELECT 이름 FROM   고객
WHERE  고객번호 NOT IN (
    SELECT 고객번호 FROM 예약
);
```

⑨상영관 번호 1번에서 상영중인 영화 가격보다 저렴한 모든 영화 재목을 구하시오

```
SELECT 영화제목, 가격 FROM   상영관
WHERE  가격 < ALL (
    SELECT 가격
    FROM   상영관
    WHERE  상영관번호 = 1
);
```

⑩예약테이블에 존재하는 극장번호를 가진 극장의 위치를 구하시오

```
SELECT 극장이름, 위치
FROM   극장 G
WHERE  EXISTS (
    SELECT 1
    FROM   예약 R
    WHERE  R.극장번호 = G.극장번호
);
```

(4)조인질의

①예약한 고객의 이름과 예약한 좌석번호, 날짜를 함께 출력하시오

```
SELECT C.이름, R.좌석번호, R.날짜
FROM   고객 C
       JOIN 예약 R ON C.고객번호 = R.고객번호;
```

②극장 이름, 상영 중인 영화 제목, 가격을 함께 출력하시오

```
SELECT G.극장이름, S.영화제목, S.가격
FROM   극장 G
       JOIN 상영관 S ON G.극장번호 = S.극장번호;
```

③ 고객이름, 극장 이름, 영화 제목을 한 번에 출력하시오

```
SELECT C.이름, G.극장이름, S.영화제목
FROM   고객 C
       JOIN 예약  R ON C.고객번호  = R.고객번호
       JOIN 상영관 S ON R.상영관번호 = S.상영관번호
       JOIN 극장  G ON R.극장번호  = G.극장번호;
```

④'잠실'에 위치한 극장에서 상영 중인 영화 제목과 가격을 출력하시오

```
SELECT G.극장이름, S.영화제목, S.가격
FROM   극장 G
       JOIN 상영관 S ON G.극장번호 = S.극장번호
WHERE  G.위치 = '잠실';
```

⑤예약정보에서 고객이름과 예약한 극장 위치를 출력하시오

```
SELECT C.이름, G.위치, R.날짜
FROM   고객 C
       JOIN 예약 R ON C.고객번호 = R.고객번호
       JOIN 극장 G ON R.극장번호 = G.극장번호;
```

⑥모든 극장과 해당 극장의 상영관 정보를 출력하시오. 상영관이 없는 극장도 포함하시오 (LEFT JOIN)

```
SELECT G.극장이름, S.영화제목, S.좌석수
FROM   극장 G
       LEFT JOIN 상영관 S ON G.극장번호 = S.극장번호;
```

⑦고객이름, 영화제목, 좌석번호, 가격을 한 번에 출력하시오(4개 테이블 조인)

```
SELECT C.이름, S.영화제목,
       R.좌석번호, S.가격
FROM   고객  C
```

```
        JOIN 예약  R ON C.고객번호  = R.고객번호
        JOIN 상영관 S ON R.상영관번호 = S.상영관번호
                   AND R.극장번호  = S.극장번호
        JOIN 극장  G ON R.극장번호  = G.극장번호;
```

(5)그룹질의

①극장별 상영관 수를 보이시오

```
SELECT 극장.극장이름, COUNT(상영관.상영관번호) AS 상영관수  FROM 극장
    JOIN 상영관 ON 극장.극장번호 = 상영관.극장번호
    GROUP BY 극장.극장이름;
```

②'잠실'에 있는 극장의 상영관을 보이시오

```
SELECT 극장.극장이름, 극장.위치, 상영관.상영관번호, 상영관.영화제목,
        상영관.가격, 상영관.좌석수   FROM 극장
     JOIN 상영관 ON 극장.극장번호 = 상영관.극장번호
     WHERE 극장.위치 = '잠실';
```

③2025년 9월 1일의 극장별 평균 관람 고객 수를 보이시오

```
SELECT 극장.극장이름,
        COUNT(DISTINCT 예약.고객번호) AS 총고객수,
        ROUND(COUNT(예약.고객번호) / COUNT(DISTINCT 상영관.상영관번호), 1) AS
평균고객수  FROM 극장
    JOIN 상영관 ON 극장.극장번호 = 상영관.극장번호
    JOIN 예약   ON 상영관.극장번호 = 예약.극장번호
            AND 상영관.상영관번호 = 예약.상영관번호
    WHERE 예약.날짜 = TO_DATE('2025/09/01', 'YYYY/MM/DD')
    GROUP BY 극장.극장이름;
```

④2025년 9월 1일에 가장 많은 고객이 관람한 영화를 보이시오

```
SELECT 상영관.영화제목, COUNT(예약.고객번호) AS 관람고객수  FROM 예약
```

```
JOIN 상영관 ON 예약.극장번호 = 상영관.극장번호                --①
                AND 예약.상영관번호 = 상영관.상영관번호
WHERE 예약.날짜 = TO_DATE('2025/09/01', 'YYYY/MM/DD')  --②
GROUP BY 상영관.영화제목                                      ---③
HAVING COUNT(예약.고객번호) = (                                ---⑤(2-결과)
    SELECT MAX(COUNT(예약.고객번호))
    FROM 예약
    JOIN 상영관 ON 예약.극장번호 = 상영관.극장번호
            AND 예약.상영관번호 = 상영관.상영관번호
    WHERE 예약.날짜 = TO_DATE('2025/09/01', 'YYYY/MM/DD')  ---④
    GROUP BY 상영관.영화제목
);
```

순서	절	하는 일
1	FROM / JOIN	예약 + 상영관 테이블 결합
2	WHERE	2025/09/01 날짜 행만 필터
3	GROUP BY	영화제목별 그룹화
4	부속질의	그룹별 최대 고객 수 계산
5	HAVING	최대 고객 수와 같은 그룹만 필터
6	SELECT	영화제목, 관람고객수 최종 출력

⑤극장번호별 예약 건수를 구하시오

```
SELECT 극장번호, COUNT(*) AS 예약건수 FROM    예약
GROUP BY 극장번호;
```

⑥극장번호별 상영관의 평균 좌석수를 구하시오

```
SELECT 극장번호, AVG(좌석수) AS 평균좌석수 FROM    상영관
GROUP BY 극장번호;
```

⑦예약건수가 2건 이상인 극장번호를 구하시오

```
SELECT 극장번호, COUNT(*) AS 예약건수 FROM    예약
GROUP BY 극장번호
HAVING  COUNT(*) >= 2;
```

⑧고객번호별 총 예약 좌석번호수와 가장 최근 예약 날짜를 구하시오

```
SELECT 고객번호,   COUNT(*) AS 예약횟수,  MAX(날짜)  AS 최근예약일
FROM    예약
GROUP BY 고객번호;
```

⑨극장번호별 상영관 수와 총 좌석 수를 구하시오

```
SELECT 극장번호, COUNT(*) AS 상영관수, SUM(좌석수) AS 총좌석수
FROM    상영관
GROUP BY 극장번호;
```

⑩극장번호별 최고 영화 가격이 10,000원을 초과하는 극장번호를 구하시오

```
SELECT 극장번호, MAX(가격) AS 최고가격 FROM    상영관
GROUP BY 극장번호
HAVING   MAX(가격) > 10000;
```

⑪고객 주소별 예약 고객 수를 구하시오(고객. 예약 조인 후 그룹)

```
SELECT C.주소, COUNT(*) AS 예약고객수
FROM    고객 C
        JOIN 예약 R ON C.고객번호 = R.고객번호
GROUP BY C.주소;
```

집계질의 : SUM,AVG, MAX,MIN,COUNT 함수를 단독으로 사용, WHERE조건과 결합해 조건부 집계 생성 가능
부속질의 : 단일행 스칼라, IN/NOT IN, ALL, EXISTS 등 4가지 유형
복합사용 , 평균, 존재 여부에 사용
조인질의 : 2개 테이블 조인,~ 여러개 조인 가능, LEFT JOIN으로 NULL 포함 조회
그룹질의 : GROUP BY, HAVING 사용

(5)DML

①각 테이블에 데이터를 삽입하는 INSERT문을 하나씩 실행시켜 보시오.

```
INSERT INTO 극장 (극장번호, 극장이름, 위치)
VALUES (4, 'CGV', '홍대');

INSERT INTO 상영관 (극장번호, 상영관번호, 영화제목, 가격, 좌석수)
VALUES (4, 1, '신나는 영화', 12000, 80);

INSERT INTO 고객 (고객번호, 이름, 주소)
VALUES (10, '이순신', '홍대');

INSERT INTO 예약 (극장번호, 상영관번호, 고객번호, 좌석번호, 날짜)
VALUES (4, 1, 10, 20, TO_DATE('2025/09/02', 'YYYY/MM/DD'));
```

②영화의 가격을 10%씩 인상하시오

③상영관 번호 1번의 가격을 15000에서 12000으로 인하하시오

```
UPDATE 상영관 SET 가격=12000 WHERE 상영관번호 =1 AND 극장번호 =1;
```

④고객번호 4번 김철수의 주소를 잠실에서 강남으로 변경하시오

```
UPDATE 고객 SET   주소 = '강남'  WHERE  고객번호 = 4;
```

⑤고객번호 9번 박영희의 예약을 취소하시오

```
DELETE FROM 예약 WHERE 고객번호=9 AND 좌석번호=48;
```

문제 21[판매원 데이터베이스]

다음 릴레이션을 보고 물음에 답하시오. Salesperson은 판매액, Order는 주문, Customer는 고객을 나타낸다. 밑줄 친 속성은 기본키이고 custname과 salesperson은 각각 Custer.name과 Salesperson.name을 참조하는 외래키이다.

Salesperson(name, age, salary)
Order(number, custname, salesperson, amount)
Customer(name, city, industrytype)

(1)교재에 나와 있는 질문

①테이블을 생성하는 CREATE 문과 데이터를 삽입하는 INSERT문을 작성하시오. 테이블의 데이터 타입은 임의로 정하고, 데이터는 다음 질의의 결과가 나오도록 삽입한다.

```
-- ① 기존 테이블 삭제 (CASCADE CONSTRAINTS로 FK 오류 방지)
DROP TABLE Orders        CASCADE CONSTRAINTS;
DROP TABLE Customer      CASCADE CONSTRAINTS;
DROP TABLE Salesperson   CASCADE CONSTRAINTS;

-- ② Salesperson 테이블 생성
CREATE TABLE Salesperson (
    name        VARCHAR2(50)    PRIMARY KEY,
    age         NUMBER(3),
    salary      NUMBER(10, 2)
);
```

```
-- ③ Customer 테이블 생성
CREATE TABLE Customer (
    name            VARCHAR2(50)    PRIMARY KEY,
    city            VARCHAR2(50),
    industrytype    VARCHAR2(50)
);

-- ④ Orders 테이블 생성 (자식 테이블 마지막)
CREATE TABLE Orders (
    number          NUMBER          PRIMARY KEY,
    custname        VARCHAR2(50)    NOT NULL,
    salesperson     VARCHAR2(50)    NOT NULL,
    amount          NUMBER(10, 2),
    FOREIGN KEY (custname)      REFERENCES Customer(name),
    FOREIGN KEY (salesperson)   REFERENCES Salesperson(name)
);
```

②모든 판매원의 이름과 급여를 보이시오. 단 중복 행은 제거한다.

```
SELECT DISTINCT name, salary FROM Salesperson;
```

③나이가 30세 미만인 판매원의 이름을 보이시오.

```
SELECT name FROM Salesperson WHERE age < 30;
```

④‘S’로 끝나는 도시에 사는 고객의 이름을 보이시오.

```
SELECT name FROM Customer WHERE city LIKE '%S';
```

⑤주문을 한 고객의 수(서로 다른 고객만)를 구하시오.

```
SELECT COUNT(DISTINCT custname) AS 고객수
FROM Orders;
```

⑥판매원 각각에 대하여 주문의 수를 계산하시오.

```
SELECT salesperson, COUNT(*) AS 주문수
FROM Orders
```

```
GROUP BY salesperson;
```

⑦‘LA’에 사는 고객으로부터 주문을 받은 판매원의 이름과 나이를 보이시오.(부속질의 사용)

```
SELECT name, age FROM Salesperson
WHERE name IN (
    SELECT salesperson
    FROM Orders
    WHERE custname IN (
        SELECT name
        FROM Customer
        WHERE city = 'LA'
    )
);
```

⑧‘LA’에 사는 고객으로부터 주문을 받은 판매원의 이름과 나이를 보이시오.(조인 사용)

```
SELECT DISTINCT Salesperson.name, Salesperson.age
FROM Salesperson
JOIN Orders    ON Salesperson.name = Orders.salesperson
JOIN Customer ON Orders.custname  = Customer.name
WHERE Customer.city = 'LA';
```

⑨두 번 이상 주문을 받은 판매원의 이름을 보이시오.

```
SELECT salesperson AS name, COUNT(*) AS 주문수  FROM Orders
GROUP BY salesperson
HAVING COUNT(*) >= 2;
```

⑩판매원 ‘TOM’의 봉급을 45,000원으로 변경하는 sql문을 작성하시오

```
UPDATE Salesperson SET salary = 45000 WHERE name = 'TOM';
```

(2)단순질의

①Salesperson테이블의 모든 데이터를 조회하시오

```
SELECT * FROM Salesperson;
```

②나이가 30세인 판매원의 이름과 나이를 조회하시오

```
SELECT name, age FROM   Salesperson  WHERE  age >= 30;
```

③급여가 3,200,000원 이상 3,500,000원 이하인 판매원을 조회하시오.

```
SELECT * FROM   Salesperson WHERE  salary BETWEEN 3200000 AND 3500000;
```

④서울에 사는 고객의 이름과 업종을 조회하시오

```
SELECT name, industrytype FROM   Customer  WHERE  city = '서울';
```

⑤주문금액이 500,000원을 초과하는 주문번호와 금액을 조회하시오.

```
SELECT number, amount FROM   Orders  WHERE  amount > 500000;
```

⑥판매원 이름을 급여 내림차순으로 정렬하여 조회하시오.

```
SELECT name, salary  FROM   Salesperson
ORDER BY salary DESC;
```

⑦IT업종 고객을 도시 오름차순으로 조회하시오.

```
SELECT * FROM   Customer  WHERE  industrytype = 'IT'
ORDER BY city ASC;
```

⑧Orders 테이블에서 고객 이름의 중복을 제거하여 조회하시오.

```
SELECT DISTINCT custname  FROM   Orders;
```

⑨이름이 ‘김’으로 시작하는 판매원을 조회하시오.

```
SELECT *  FROM   Salesperson  WHERE  name LIKE '김%';
```

⑩주문 금액을 기준으로 상위 3건만 조회하시오

```
-- Oracle
SELECT * FROM (
    SELECT * FROM Orders
    ORDER BY amount DESC
) WHERE ROWNUM <= 3;
```

(3)부속질의

①평균 급여보다 급여가 높은 판매원의 이름과 급여를 조회하시오

```
SELECT name, salary
FROM    Salesperson
WHERE   salary > (
    SELECT AVG(salary) FROM Salesperson
);
```

②주문을 한 번만이라도 받은 판매원의 이름을 조회하시오

```
SELECT name
FROM    Salesperson
WHERE   name IN (
    SELECT salesperson FROM Orders
);
```

③한 번도 주문을 받지 않은 판매원의 이름을 조회하시오.

```
SELECT name
FROM    Salesperson
WHERE   name NOT IN (
    SELECT salesperson FROM Orders   );
```

④서울에 사는 고객이 주문한 모든 주문번호와 금액을 조회하시오.

```
SELECT number, amount   FROM    Orders
WHERE   custname IN (
    SELECT name
    FROM    Customer
    WHERE   city = '서울'
);
```

⑤가장 높은 주문금액을 처리한 판매원의 이름과 급여를 조회하시오.

```
SELECT name, salary   FROM    Salesperson
WHERE   name = (
    SELECT salesperson
    FROM    Orders
    WHERE   amount = (
        SELECT MAX(amount) FROM Orders
    )
);
```

⑥주문금액이 모든 주문의 평균보다 큰 금액을 주문받은 판매원 이름을 중복 없이 조회하시

오.

```
SELECT DISTINCT salesperson
FROM    Orders
WHERE   amount > (
    SELECT AVG(amount) FROM Orders
);
```

⑦IT 업종 고객에게 주문을 받은 적이 있는 판매원의 이름과 나이를 조회하시오.

```
SELECT name
FROM    Salesperson S
WHERE   EXISTS (
    SELECT 1
    FROM    Orders O
            JOIN Customer C ON O.custname = C.name
    WHERE   O.salesperson   = S.name
      AND   C.industrytype = 'IT'
);
```

⑧최고 급여 판매원보다 판매금액이 높은 판매원의 이름과 나이를 조회하시오.

```
-- Step 1: 최고 급여 판매원의 총 판매금액을 부속질의로 구함
-- Step 2: 다른 판매원의 총 판매금액이 그보다 큰 경우 조회

SELECT S.name, S.age   FROM    Salesperson S
       JOIN Orders O ON S.name = O.salesperson
GROUP BY S.name, S.age
HAVING SUM(O.amount) > (
    SELECT SUM(O2.amount)    FROM    Orders O2
    WHERE   O2.salesperson = (
        SELECT name
        FROM    Salesperson
        WHERE   salary = (SELECT MAX(salary) FROM Salesperson)
    )  );
```

⑨‘홍길동’ 고객의 모든 주문보다 금액이 큰 주문번호와 금액을 조회하시오.

```
SELECT number, amount FROM    Orders
```

```
WHERE   amount > ALL (
    SELECT amount
    FROM    Orders
    WHERE   custname = '홍길동'
);
```

문제 분석

"최고 급여 판매원보다 **판매금액(Orders.amount 합계)**이 높은 판매원"
→ **최고 급여 판매원** 을 먼저 구한 뒤 (MAX salary → 최수연, 4,000,000)
→ 판매원별 **총 판매금액 SUM(amount)** 이 그보다 큰 판매원을 찾아야 함
→ **두 테이블(Salesperson, Orders)을 연결 + 집계 + 부속질의** 필요

오답 — 잘못된 SQL

× 오답 1 — salary를 amount와 직접 비교

```
SELECT name, age
FROM   Salesperson
WHERE  salary > (SELECT MAX(salary) FROM Salesperson);
```

급여(salary)끼리 비교하는 쿼리 → **판매금액(amount)과 무관**하며, MAX보다 큰 값은 존재할 수 없어 항상 결과 없음.

× 오답 2 — 집계 없이 amount 단순 비교

```
SELECT S.name, S.age
FROM   Salesperson S
       JOIN Orders O ON S.name = O.salesperson
WHERE  O.amount > (SELECT MAX(salary) FROM Salesperson);
```

개별 주문 1건의 금액과 비교 → **총 판매금액 SUM이 아닌 단건 비교**라 의미가 다름. 최고 급여(4,000,000)보다 큰 단건 주문은 없으므로 결과 없음.

문제의 핵심 — 3단계 중첩 부속질의

1. `MAX(salary)` → 최고 급여 = 4,000,000 (최수연)
2. 최수연의 `SUM(amount)` → 총 판매금액 = 1,000,000
3. 다른 판매원의 `SUM(amount)` > 1,000,000 인 판매원 조회

⑩주문을 한 번도 하지 않은 고객의 이름과 도시를 조회하시오.

```
SELECT name, city   FROM    Customer
WHERE   name NOT IN (
    SELECT custname FROM Orders
);
```

(4)조인 질의

①주문번호 , 고객이름, 담당 판매원 이름, 주문 금액을 조회하시오

```
SELECT O.number, C.name AS 고객명, S.name AS 판매원명, O.amount
FROM    Orders       O
        JOIN Customer    C ON O.custname    = C.name
        JOIN Salesperson S ON O.salesperson = S.name;
```

②서울 고객이 주문한 내역이세 담당 판매원 이름과 금액을 조회하시오.

```
SELECT O.salesperson, O.amount, C.city
FROM    Orders    O
        JOIN Customer C ON O.custname = C.name
WHERE   C.city = '서울';
```

③판매원 이름, 급여, 담당한 주문 금액을 함께 조회하시오. 주문이 없는 판매원도 포함하시오

```
SELECT S.name, S.salary, O.amount
FROM    Salesperson S
        LEFT JOIN Orders O ON S.name = O.salesperson;
```

④ IT업종 고객의 주문을 처리한 판매원 이름과 주문 금액을 조회하시오.

```
SELECT O.salesperson, O.amount, C.industrytype
FROM    Orders     O
        JOIN Customer C ON O.custname = C.name
WHERE   C.industrytype = 'IT';
```

⑤판매원 급여가 3,200,000원 이상인 판매원이 처리한 주문의 고객 이름과 금액을 조회하시오.

```
SELECT S.name AS 판매원, O.custname, O.amount   FROM    Salesperson S
        JOIN Orders O ON S.name = O.salesperson
WHERE   S.salary >= 3200000;
```

⑥고객 이름, 도시, 주문 금액을 조회하되 주문이 없는 고객도 포함하시오.

```
SELECT C.name, C.city, O.amount   FROM    Customer C
        LEFT JOIN Orders O ON C.name = O.custname;
```

⑦주문 금액이 500,000원 초과인 주문의 고객 도시와 판매원 급여를 함께 조회하시오.

```
SELECT C.city, S.salary, O.amount
FROM    Orders       O
```

```
        JOIN Customer    C ON O.custname    = C.name
        JOIN Salesperson S ON O.salesperson = S.name
WHERE   O.amount > 500000;
```

⑧'김철 수 판매원이 처리한 주문의 고객 이름과 도시를 조회하시오.

```
SELECT C.name, C.city, O.amount
FROM    Orders     O
        JOIN Customer C ON O.custname = C.name
WHERE   O.salesperson = '김철수';
```

⑨같은 도시에 사는 고객끼리 조합을 구하시오(셀프조인).

```
SELECT A.name AS 고객1, B.name AS 고객2, A.city
FROM    Customer A
        JOIN Customer B ON A.city = B.city
WHERE   A.name < B.name; -- 중복 제거
```

⑩판매원 이름, 나이, 처리한 주문의 고객 업종을 모두 출력하시오(3개 테이블 조인)

```
SELECT S.name, S.age, C.industrytype, O.amount
FROM    Salesperson S
        JOIN Orders     O ON S.name       = O.salesperson
        JOIN Customer   C ON O.custname = C.name;
```

(5)집계질의

①전체주문의 총 금액과 평균 금액을 구하시오.

```
SELECT SUM(amount) AS 총주문금액,  AVG(amount) AS 평균주문금액    FROM    Orders;
```

②판매원별 처리한 주문 건수와 총 주문 금액을 구하시오.

```
SELECT salesperson,  COUNT(*) AS 주문건수, SUM(amount) AS 총금액
FROM    Orders
GROUP BY salesperson;
```

③고객별 총 주문금액이 1,000,000원 이상인 고객만 조회하시오.

```
SELECT custname, SUM(amount) AS 총주문금액  FROM    Orders
```

```
GROUP BY custname
HAVING  SUM(amount) >= 1000000;
```

④판매원의 최고 급여, 최저급여, 평균 급여를 구하시오.

```
SELECT MAX(salary) AS 최고급여,
       MIN(salary) AS 최저급여,
       AVG(salary) AS 평균급여
FROM   Salesperson;
```

⑤2건 이상 주문을 처리한 판매원과 주문 건수를 조회하시오

```
SELECT salesperson, COUNT(*) AS 주문건수  FROM   Orders
GROUP BY salesperson
HAVING  COUNT(*) >= 2;
```

⑥도시별 고객 수를 구하시오.

```
SELECT city, COUNT(*) AS 고객수   FROM   Customer
GROUP BY city
ORDER BY 고객수 DESC;
```

⑦판매원별 평균 주문 금액이 가장 높은 판매원을 조회하시오.

```
-- Oracle
SELECT * FROM (
    SELECT salesperson, AVG(amount) AS 평균금액
    FROM   Orders
    GROUP BY salesperson
    ORDER BY 평균금액 DESC
) WHERE ROWNUM = 1;
```

⑧업종별 고객 수와 업종 목록을 구하시오.

```
SELECT industrytype, COUNT(*) AS 고객수  FROM   Customer
GROUP BY industrytype;
```

⑨판매원별 총 주문 금액을 구하되, 조인을 통해 판매원 급여도 함께 출력하시오.

```
SELECT S.name, S.salary, SUM(O.amount) AS 총주문금액  FROM   Salesperson S
```

```
        JOIN Orders O ON S.name = O.salesperson
GROUP BY S.name, S.salary
ORDER BY 총주문금액 DESC;
```

⑩도시별, 업종별 고객 수를 구하시오.(다중 GROUP BY)

```
SELECT city, industrytype,  COUNT(*) AS 고객수  FROM   Customer
GROUP BY city, industrytype
ORDER BY city;
```

문제 22

[기업 프로젝트 데이터베이스]

다음 릴레이션을 보고 물음에 답하시오. Employee는 사원, Department는 부서, Project는 프로젝트, Works는 사원이 프로젝트에 참여한 내용을 나타낸다. 한 사원이 여러 프로젝트에서 일하거나 한 프로젝트에 여러 사원이 일할 수 있다. hours_worked속성은 각 사원이 각 프로젝트에서 일한 시간을 나타낸다. 밑줄 친 속성은 기본키이다.

Employee(<u>empno,</u> name, phoneno, address, sex, position, depno)
Department(<u>depno</u>, deptname, ,manager)
Project(<u>projno</u>, projname, deptno)
Works(<u>empno</u>, <u>projno</u>, hours_worked)

(1)교재에 있는 질의

①테이블을 생성하는 CREATE문과 데이터를 삽입하는 INSERT문을 작성하시오. 테이블의 데이터 타입은 임의로 정하고, 데이터는 다음 질의의 결과가 나오도록 삽입한다.

```
기존 테이블을 삭제한다.

drop table if exists works;
drop table if exists project;
drop table if exists employee;
drop table if exists department;

CREATE TABLE Department
(
deptno int not null,
deptname varchar2(20),
manager int,
```

```
primary key(deptno)
);
CREATE TABLE Employee
(
empno int not null,
name varchar2(20),
phoneno int,
address varchar2(20),
sex varchar2(20),
position varchar2(20),
deptno int,
primary key(empno),
foreign key(deptno) references Department(deptno)
);

CREATE TABLE Project
(
projno int not null,
projname varchar2(20),
deptno int,
primary key(projno),
foreign key(deptno) references Department(deptno)
);

CREATE TABLE Works
(
projno int not null,
empno int not null,
hours_worked int,
PRIMARY key(projno, empno),
foreign key(projno) references Project(projno),
foreign key(empno) references Emplyoee(empno)
);
```

②모든 사원의 이름을 보이시오

```
SELECT name FROM Employee;
```

③여자 사원의 이름을 보이시오

```
SELECT name FROM Employee
WHERE sex LIKE '여';
```

④팀장(manager)의 이름을 보이시오

```
SELECT name FROM Employee
WHERE empno IN (SELECT manager FROM Department);
```

⑤'IT'부서에서 일하는 사원의 이름과 주소를 보이시오

```
SELECT name, address FROM Employee, Department
WHERE Employee.deptno=Department.deptno and deptname LIKE 'IT';
```

⑥'홍길동' 팀장(manager)부서에서 일하는 사원의 수를 보이시오

```
SELECT count(name) FROM Employee
WHERE deptno IN (SELECT Department.deptno FROM Employee, Department
                 WHEREEmployee.empno=Department.manager
                  AND name LIKE '홍길동');
```

⑦사원들이 일한 시간 수를 부서별, 사원 이름별 오름차순으로 보이시오

```
SELECT deptno, name, SUM(hours_worked) FROM Employee, Works
WHERE Emplyee.empno=Works.empno
GROUP BY  deptno, name
ORDER BY  deptno, name;
```

⑧2명 이상의 사원이 참여한 프로젝트의 번호, 이름, 사원의 수를 보이시오

```
SELECT  Project.projno, Project.projname, count(name) 사원수
FROM  Employee, Project
WHERE  Project.deptno=Employee.deptno
GROUP BY  Project.projno, Project.projname
HAVING COUNT(*) >= 2;
```

⑨3명 이상의 사원이 있는 부서의 사원 이름을 보이시오

```
SELECT name  FROM Employee, Department
WHERE Employee.deptno=Department.deptno and
        deptname = (SELECT deptname FROM Employee, Department
                    WHERE Employee.deptno=Department.deptno
                     GROUP BY deptname
                     HAVING COUNT(name) >= 3)
```

(2)단순질의

①전체 직원의 이름과 직위를 조회하시오.

```
SELECT name, position  FROM Employee;
```

②IT부서(deptno=1) 소속 직원의 이름과 연락처를 조회하시오.

```
SELECT name, phoneno FROM   Employee
WHERE  deptno = 1;
```

③여성 직원의 이름과 주소를 조회하시오.

```
SELECT name, address  FROM   Employee
WHERE  sex = '여';
```

④직위가 ‘사원’이거나 ‘대리’인 직원의 이름과 부서번호를 조회하시오.

```
SELECT name, position, deptno  FROM   Employee
WHERE  position IN ('사원', '대리');
```

⑤근무시간이 30시간 이상 45시간 이하인 Works데이터를 조회하시오.

```
SELECT *  FROM   Works
WHERE  hours_worked BETWEEN 30 AND 45;
```

⑥이름에 ‘수’가 포함된 직원을 조회하시오.

```
SELECT *  FROM   Employee
WHERE  name LIKE '%수%';
```

⑦프로젝트 목록을 프로젝트명 오름차순으로 조회하시오.

```
SELECT projno, projname  FROM   Project
ORDER BY projname ASC;
```

⑧Works에서 101번 직원의 프로젝트별 근무시간을 조회하시오.

```
SELECT projno, hours_worked  FROM   Works
WHERE  empno = 101;
```

⑨Works에서 참여한 직원번호를 중복없이 조회하시오.

```
SELECT DISTINCT empno  FROM   Works;
```

⑩주소가 ‘서울시 강남구’ 가 아닌 직원의 이름과 주소를 조회하시오.

```
SELECT name, address  FROM   Employee
WHERE  address != '서울시 강남구';
```

(3)복합질의

①프로젝트에 참여한 직원의 이름을 조회하시오.

```
SELECT name  FROM   Employee
WHERE  empno IN ( SELECT empno FROM Works);
```

②평균 근무시간(AVG)보다 많이 근무한 직원번호와 근무시간을 조회하시오.

```
SELECT empno, projno, hours_worked  FROM   Works
WHERE  hours_worked > ( SELECT AVG(hours_worked) FROM Works);
```

③부서장(manager)인 직원의 이름과 직위를 조회하시오(EXITS)

```
SELECT name, position  FROM   Employee E
WHERE  EXISTS (
               SELECT 1
                FROM   Department D
                WHERE  D.manager = E.empno  );
```

④가장 많은 시간을 근무한 직원의 사원번호와 근무시간을 조회하시오.

```
SELECT empno, projno, hours_worked FROM   Works
WHERE  hours_worked = (
    SELECT MAX(hours_worked) FROM Works );
```

⑤IT부서(depno=1)소속 직원이 참여한 프로젝트 번호를 조회하시오.

```
SELECT DISTINCT projno FROM   Works
WHERE  empno IN (
    SELECT empno
    FROM   Employee
    WHERE  deptno = 1
);
```

⑥여성 직원이 참여한 프로젝트명을 조회하시오(3단계 부속 질의)

```
SELECT DISTINCT projname
FROM    Project
WHERE   projno IN (
    SELECT projno    FROM    Works
    WHERE   empno IN (
        SELECT empno
        FROM    Employee
        WHERE   sex = '여'
    )
);
```

⑦2개 이상의 프로젝트에 참여한 직원의 이름을 조회하시오.

```
 SELECT name FROM    Employee
 WHERE   empno IN (
    SELECT empno
    FROM    Works
    GROUP BY empno
    HAVING COUNT(*) >= 2
);
```

⑧프로젝트1 참여자의 모든 근무시간보다 많이 근무한 데이터를 조회하시오.(ALL)

```
SELECT empno, projno, hours_worked
FROM    Works
WHERE   hours_worked > ALL (
    SELECT hours_worked
    FROM    Works
    WHERE   projno = 1
);
```

⑨부서장이 아닌 직원의 이름과 직위를 조회하시오.(NOT EXITS)

```
SELECT name, position
FROM    Employee E
WHERE   NOT EXISTS (
    SELECT 1
    FROM    Department D
    WHERE   D.manager = E.empno
);
```

⑩IT부서 직원이 한 명이라도 참여한 프로젝트명을 조회하시오. (EXITS)

```
SELECT projname  FROM   Project P
WHERE  EXISTS (
    SELECT 1
    FROM   Works W

           JOIN Employee E ON W.empno = E.empno
    WHERE  W.projno = P.projno
      AND  E.deptno = 1
);
```

(4)집계질의

①전체 직원수를 구하시오.

```
SELECT COUNT(*) AS 전체직원수
FROM   Employee;
```

②Works 테이블의 전체 최대,최소 평균 근무시간을 구하시오.

```
SELECT MAX(hours_worked) AS 최대,  MIN(hours_worked) AS 최소,
       AVG(hours_worked) AS 평균  FROM   Works;
```

③부서별 직원 수를 구하시오.

```
SELECT deptno, COUNT(*) AS 직원수
FROM   Employee
GROUP BY deptno
ORDER BY deptno;
```

④프로젝트별 총 근무시간과 참여 인원을 구하시오.

```
SELECT projno,
       SUM(hours_worked) AS 총근무시간,
       COUNT(*) AS 참여인원
FROM   Works
GROUP BY projno;
```

⑤총 근무시간이 80시간 이상인 프로젝트를 조회하시오.

```
SELECT projno, SUM(hours_worked) AS 총시간
FROM    Works
GROUP BY projno
HAVING  SUM(hours_worked) >= 80;
```

⑥성별 직원 수와 비율을 구하시오.

```
SELECT sex, COUNT(*) AS 직원수  FROM    Employee
GROUP BY sex;
```

⑦직원별 총 근무시간을 구하고 내림차순 정렬하시오.

```
SELECT empno, SUM(hours_worked) AS 총근무시간 FROM    Works
GROUP BY empno
ORDER BY 총근무시간 DESC;
```

⑧부서별, 직위별 직원 수를 구하시오(다중 GROUP BY).

```
SELECT deptno, position, COUNT(*) AS 직원수 FROM    Employee
GROUP BY deptno, position
ORDER BY deptno;
```

⑨프로젝트별 평균 근무시간이 40시간 미만인 프로젝트를 조회하시오.

```
SELECT projno, AVG(hours_worked) AS 평균시간
FROM    Works
GROUP BY projno
HAVING  AVG(hours_worked) < 40;
```

⑩총 근무시간이 가장 많은 직원의 사원번호와 총 시간을 구하시오.

```
-- Oracle
SELECT * FROM (
    SELECT empno, SUM(hours_worked) AS 총시간
    FROM    Works
    GROUP BY empno
    ORDER BY 총시간 DESC
) WHERE ROWNUM = 1;

[수행순서]
①안쪽 서브쿼리부터 실행 : GROUP BY+ORDER BY 총시간 DESC
②바같쪽 쿼리 실행 : ROWNUM=1인 행만 추출
```

반드시 서브쿼리를 써야 한다.

ROWNUM을 서브쿼리 없이 직접 쓰면 ORDER BY 가 실행되기 전에 번호가 붙어 정렬순서가 보장되지 않는다.

(5)조인 질의

①직원 이름과 소속 부서명을 함께 조회하시오.

```
SELECT E.name, D.deptname
FROM   Employee   E
       JOIN Department D ON E.deptno = D.deptno;
```

②직원 이름, 참여 프로젝트명, 근무시간을 조회하시오.

```
SELECT E.name, P.projname, W.hours_worked
FROM   Employee E
       JOIN Works   W ON E.empno  = W.empno
       JOIN Project P ON W.projno = P.projno;
```

③부서장 이름과 담당 부서명을 조회하시오.

```
SELECT E.name AS 부서장, D.deptname
FROM   Department D
       JOIN Employee   E ON D.manager = E.empno;
```

④IT부서 소속 직원이 참여한 프로젝트명과 근무시간을 조회하시오.

```
SELECT E.name, P.projname, W.hours_worked
FROM   Employee   E
       JOIN Department D ON E.deptno  = D.deptno
```

```
        JOIN Works       W ON E.empno    = W.empno
        JOIN Project     P ON W.projno   = P.projno
WHERE   D.deptname = 'IT부서';
```

⑤프로젝트별 참여하지 않은 직원도 포함하여 이름과 프로젝트명을 조회하시오.

```
SELECT D.deptname, P.projname, SUM(W.hours_worked) AS 총근무시간
FROM    Works  W
        JOIN Project     P ON W.projno = P.projno
        JOIN Department D ON P.deptno = D.deptno
GROUP BY D.deptname, P.projname;
```

⑥프로젝트에 참여하지 않은 직원도 포함하여 이름과 프로젝트명을 조회하시오.(LEFT JOIN)

```
SELECT E.name, P.projname, W.hours_worked
FROM    Employee E
        LEFT JOIN Works    W ON E.empno  = W.empno
        LEFT JOIN Project P ON W.projno = P.projno;
```

⑦직원이름, 부서명, 프로젝트명, 근무시간을 모두 출력하시오.

```
SELECT E.name, D.deptname,  P.projname, W.hours_worked
FROM    Employee    E
        JOIN Department D ON E.deptno   = D.deptno
        JOIN Works       W ON E.empno    = W.empno
        JOIN Project     P ON W.projno   = P.projno;
```

⑧근무시간이 40시간 이상인 직원의 이름, 부서명, 프로젝트명을 조회하시오.

```
SELECT E.name, D.deptname, P.projname, W.hours_worked
FROM    Employee    E
        JOIN Department D ON E.deptno   = D.deptno
        JOIN Works       W ON E.empno    = W.empno
        JOIN Project     P ON W.projno   = P.projno
WHERE   W.hours_worked >= 40;
```

⑨여성 직원이 참여한 프로젝트명과 담당 부서명을 조회하시오.

```
SELECT DISTINCT E.name, P.projname, D.deptname
FROM    Employee    E

        JOIN Works       W ON E.empno  = W.empno
        JOIN Project     P ON W.projno = P.projno
        JOIN Department D ON P.deptno = D.deptno
WHERE   E.sex = '여';
```

⑩부서별 총 근무시간이 가장 많은 부서명과 시간을 조회하시오.

```
-- Oracle
SELECT * FROM (
    SELECT D.deptname,
           SUM(W.hours_worked) AS 총근무시간
    FROM    Employee    E
           JOIN Department D ON E.deptno = D.deptno
           JOIN Works       W ON E.empno  = W.empno
    GROUP BY D.deptname
    ORDER BY 총근무시간 DESC
) WHERE ROWNUM = 1;
```